

Projet — AutonomicMesh

Middleware distribué auto-géré avec gRPC, IPC et la boucle MAPE-K

Fiche du projet

Durée : 3 semaines (travail en équipe de 2 ou 3 étudiants)

Technologies : Java 21, gRPC (Protobuf), Java IPC (sockets/pipes), MAPE-K loop

Concepts couverts: Architecture RPC, IPC, Autonomic Computing, Middleware réparti

Rendu final : Code source (GitHub/ZIP) + rapport PDF + démonstration (10 min)

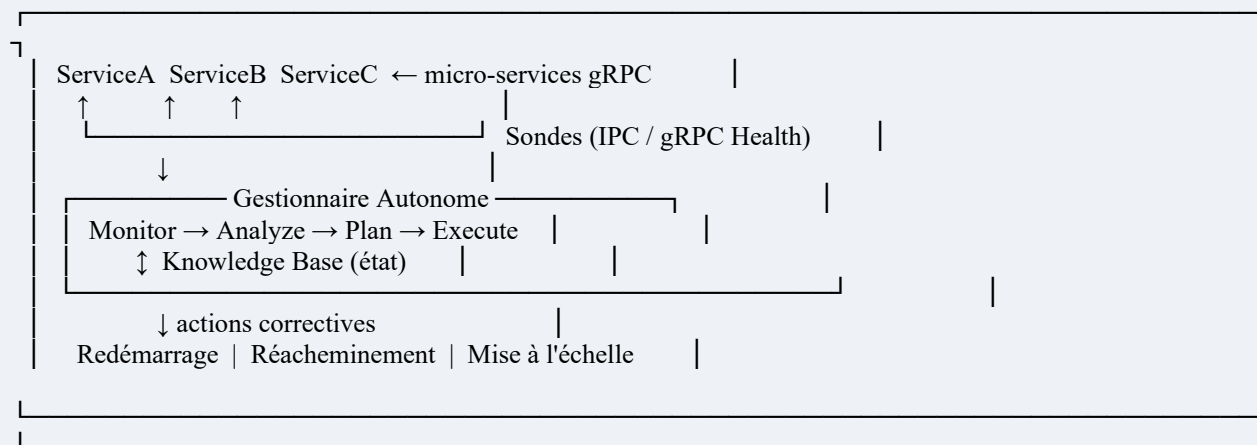
Date limite : Fin de la semaine 3 (selon calendrier affiché sur Moodle)

1. Contexte et mise en situation

Les infrastructures numériques modernes sont trop complexes pour être administrées manuellement. Le paradigme de l'informatique autonome (Autonomic Computing), proposé par IBM en 2001, répond à ce défi avec des systèmes capables de se configurer, se guérir, s'optimiser et se protéger automatiquement grâce à la boucle de contrôle MAPE-K (Monitor → Analyze → Plan → Execute, Knowledge base).

Dans ce projet, vous construisez AutonomicMesh : un middleware distribué composé de micro-services Java communiquant via gRPC, supervisé par un gestionnaire autonome qui implémente la boucle MAPE-K. Quand un service tombe en panne ou se dégrade, le gestionnaire le détecte, décide d'une action corrective et la réalise sans intervention humaine (redémarrage, réacheminement du trafic, mise à l'échelle).

Architecture MAPE-K cible



- gRPC (Protobuf) : communication inter-services + health check
- IPC (sockets) : canal de supervision entre sondes et gestionnaire
- MAPE-K : boucle autonome (cycle toutes les 10 secondes)

2. Planification — Trois semaines de travail

Objectifs semaine 1

- Définir les contrats Protobuf des services et du health-check
- Implémenter 3 micro-services gRPC Java (Calcul, Stockage, Notification)
- Implémenter des sondes IPC (sockets) qui mesurent la latence et la disponibilité
- Déployer et tester la communication inter-services

Tâche 1.1 — Définition Protobuf

Précisions — contrat Protobuf

Le fichier `mesh.proto` doit contenir les messages et services nécessaires aux trois micro-services. Les noms proposés peuvent être conservés.

Chaque service doit exposer au minimum une opération métier et un HealthCheck. Le HealthCheck doit permettre à une sonde de distinguer SERVING, DEGRADED et DOWN.

Après modification du fichier `.proto`, régénérez les classes Java avec Maven et vérifiez que le projet compile avant de continuer.

mesh.proto — à compléter selon votre conception

```
// mesh.proto - définition commune
syntax = "proto3";
option java_package = "ca.usherbrooke.ift605.mesh.proto";
option java_outer_classname = "MeshProto";

// Service de calcul
service CalcService {
    rpc Compute(CalcRequest) returns (CalcResponse);
    rpc HealthCheck(HealthReq) returns (HealthResp);
}
message CalcRequest { double a = 1; double b = 2; string op = 3; }
message CalcResponse { double result = 1; int64 latencyMs = 2; }

// Health check universel
message HealthReq { string serviceName = 1; }
message HealthResp { string status = 1; int64 uptimeMs = 2; double cpuLoad = 3; }

// Service de notification (pub interne)
service NotifService {
    rpc Notify(NotifRequest) returns (NotifResponse);
    rpc HealthCheck(HealthReq) returns (HealthResp);
}
message NotifRequest { string event = 1; string payload = 2; }
message NotifResponse { bool accepted = 1; }
```

Tâche 1.2 — Micro-service CalcService

Travail attendu pour les trois micro-services

1. CalcService : exécuter une opération de calcul et retourner le résultat ainsi qu'une mesure de latence.
2. StorageService : proposer une opération simple de stockage/récupération définie par votre équipe.

3. NotifService : recevoir un événement et confirmer sa prise en charge.

Chaque service doit être lancé comme un processus Java indépendant sur un port différent.

Validation : un client gRPC doit pouvoir appeler chacun des trois services et obtenir une réponse correcte.

CalcServiceImpl.java — squelette

```
public class CalcServiceImpl extends CalcServiceGrpc.CalcServiceImplBase {
    private final long startTime = System.currentTimeMillis();

    @Override
    public void compute(CalcRequest req, StreamObserver<CalcResponse> resp) {
        long t0 = System.currentTimeMillis();
        double result = switch (req.getOp()) {
            case "+" -> req.getA() + req.getB();
            case "-" -> req.getA() - req.getB();
            case "*" -> req.getA() * req.getB();
            case "/" -> req.getB() != 0 ? req.getA() / req.getB()
                                   : Double.NaN;
            default -> throw new StatusException(Status.INVALID_ARGUMENT);
        };
        resp.onNext(CalcResponse.newBuilder()
            .setResult(result)
            .setLatencyMs(System.currentTimeMillis() - t0)
            .build());
        resp.onCompleted();
    }

    @Override
    public void healthCheck(HealthReq req, StreamObserver<HealthResp> resp) {
        // TODO : retourner SERVING / DEGRADED / DOWN + uptime + charge CPU
        throw new UnsupportedOperationException("À implémenter");
    }
}
```

Tâche 1.3 — Sonde IPC (socket)

Format minimal d'une mesure de santé

Chaque MesureSante transmise au gestionnaire doit contenir au minimum : service, timestamp, statut, latenceMs, cpuLoad et uptimeMs.

La sonde effectue un HealthCheck toutes les 5 secondes. Elle transmet ensuite la mesure au Monitor par socket TCP sur le port IPC configuré.

Si l'appel gRPC échoue, la sonde doit produire une mesure DOWN plutôt que terminer brutalement.

Validation semaine 1 : les trois services démarrent, les sondes produisent des mesures périodiques et le Monitor peut recevoir au moins une mesure de chaque service.

Chaque sonde tourne dans un thread et envoie périodiquement (toutes les 5 s) une mesure de disponibilité au gestionnaire autonome via un socket TCP local (port 9090).

Sonde.java — squelette

```
public class Sonde implements Runnable {
    private final String service;
    private final int   grpcPort;    // port du service à surveiller
    private final int   ipcPort;     // port de rapport vers le gestionnaire
}
```

```

@Override
public void run() {
    while (true) {
        MesureSante m = pinger();           // appel gRPC healthCheck
        envoyerIPC(m);                       // rapport via socket
        Thread.sleep(5000);
    }
}
// TODO : implémenter pinger() + envoyerIPC()
}

```

S2

Boucle MAPE-K — Gestionnaire Autonome

Semaine 2

Objectifs semaine 2

- Implémenter les 4 composants de la boucle MAPE-K
- Maintenir une base de connaissances (Knowledge Base) de l'état du système
- Définir des politiques d'auto-guérison (redémarrage, réacheminement)
- Simuler des pannes et valider la réaction autonome

Tâche 2.1 — Monitor (collecte)

Responsabilité du Monitor

Le Monitor ne décide pas des actions correctives. Il reçoit les mesures JSON, les valide et met à jour la KnowledgeBase.

Une nouvelle mesure remplace l'état courant du service concerné tout en permettant de conserver, si vous le souhaitez, un petit historique utile au rapport.

Monitor.java — squelette

```

public class Monitor {
    // Reçoit les mesures des sondes IPC et les stocke dans la KB
    private final KnowledgeBase kb;
    private ServerSocket server;

    public void demarrer(int port) throws IOException {
        server = new ServerSocket(port);
        while (true) {
            Socket s = server.accept();
            // Lire MesureSante (JSON), parser, stocker dans kb
            // TODO
        }
    }
}

```

Tâche 2.2 — Analyze (analyse)

Règles minimales d'analyse

PANNE_TOTALE : statut DOWN.

DEGRADATION : latence supérieure à 500 ms.

SURCHARGE : cpuLoad supérieur à 0,90.

Ces seuils constituent la configuration minimale demandée. Vous pouvez les rendre configurables, mais expliquez tout changement dans le rapport.

Analyzer.java — squelette

```
public class Analyzer {
    private final KnowledgeBase kb;

    // Retourne une liste de symptômes détectés
    public List<Symptome> analyser() {
        List<Symptome> symptomes = new ArrayList<>();
        for (var entry : kb.getMesures().entrySet()) {
            MesureSante m = entry.getValue();
            if (m.statut.equals("DOWN"))
                symptomes.add(new Symptome(entry.getKey(),
TypeSymptome.PANNE_TOTALE));
            else if (m.latenceMs > 500)
                symptomes.add(new Symptome(entry.getKey(),
TypeSymptome.Degradation));
            // TODO : ajouter règle pour cpuLoad > 0.9
        }
        return symptomes;
    }
}
```

Tâche 2.3 — Plan & Execute

Comportement attendu de Plan et Execute

Planner transforme les symptômes en actions sans exécuter directement ces actions.

Executor réalise réellement au moins deux types d'actions correctives parmi : redémarrage, réacheminement et mise à l'échelle.

Une simple impression console ne suffit pas pour valider une action corrective : l'effet doit être observable dans le système ou dans son état de routage.

Validation semaine 2 : provoquer une panne et une dégradation, puis montrer dans les logs la séquence Monitor → Analyze → Plan → Execute.

Planner.java & Executor.java — squelettes

```
public class Planner {
    public List<ActionCorrective> planifier(List<Symptome> symptomes) {
        return symptomes.stream().map(s -> switch (s.type) {
            case PANNE_TOTALE -> new ActionCorrective(s.service,
TypeAction.REDEMARRER);
            case DEGRADATION -> new ActionCorrective(s.service,
TypeAction.REACHEMINEMENT);
            case SURCHARGE -> new ActionCorrective(s.service,
TypeAction.MISE_A_ECHELLE);
        }).toList();
    }
}

public class Executor {
    public void executer(ActionCorrective action) {
        // TODO selon action.type :
        // REDEMARRER → relancer le processus gRPC (ProcessBuilder)
        // REACHEMINEMENT → modifier la KB pour rediriger vers service de secours
        // MISE_A_ECHELLE → démarrer une instance supplémentaire du service
    }
}
```

```
}
```

Tâche 2.4 — Knowledge Base & boucle principale

KnowledgeBase.java & GestionnaireAutonome.java — squelettes

```
public class KnowledgeBase {
    private final ConcurrentHashMap<String, MesureSante> mesures = new
ConcurrentHashMap<>();
    private final ConcurrentHashMap<String, Integer>      nbPannes = new
ConcurrentHashMap<>();
    private final ConcurrentHashMap<String, String>      routage  = new
ConcurrentHashMap<>();
    // getters / setters thread-safe
}

public class GestionnaireAutonome {
    private final Monitor  monitor  = new Monitor(kb);
    private final Analyzer analyzer = new Analyzer(kb);
    private final Planner  planner  = new Planner();
    private final Executor executor = new Executor(kb);
    private final KnowledgeBase kb  = new KnowledgeBase();

    public void demarrer() {
        new Thread(() -> monitor.demarrer(9090)).start();
        ScheduledExecutorService sched = Executors.newScheduledThreadPool(1);
        sched.scheduleAtFixedRate(() -> {
            var symptomes = analyzer.analyser();
            var actions    = planner.planifier(symptomes);
            actions.forEach(executor::executer);
        }, 0, 10, TimeUnit.SECONDS);    // cycle MAPE-K toutes les 10 s
    }
}
```

S3

Tests de chaos, analyse & rapport

Semaine 3

Objectifs semaine 3

- Implémenter des tests de chaos (chaos engineering) : tuer des services aléatoirement
- Mesurer le MTTR (Mean Time To Recovery) avant/après MAPE-K
- Analyser les limites et faux positifs du gestionnaire autonome
- Rédiger le rapport et préparer la démonstration

Tâche 3.1 — ChaosMonkey (injection de pannes)

Consignes pour les tests de chaos

Le ChaosMonkey doit agir uniquement sur les processus de votre projet et dans votre environnement local.

Pour chaque panne injectée, journalisez au minimum : service ciblé, heure de la panne, heure de détection, action choisie et heure de récupération.

Réalisez plusieurs essais afin de calculer un MTTR moyen exploitable dans le rapport.

ChaosMonkey.java — squelette

```
public class ChaosMonkey {
    // Tue un service aléatoire toutes les 30 secondes
}
```

```

public void demarrer(List<String> services) {
    ScheduledExecutorService sched = Executors.newScheduledThreadPool(1);
    sched.scheduleAtFixedRate(() -> {
        String cible = services.get(new Random().nextInt(services.size()));
        System.out.println("[Chaos] Arrêt de " + cible);
        // Envoyer SIGTERM au processus gRPC correspondant
        // Mesurer le temps de récupération automatique par le Gestionnaire
    }, 30, 30, TimeUnit.SECONDS);
}
}

```

Tâche 3.2 — Métriques à mesurer et analyser

Métriques obligatoires dans le rapport

- MTTR (Mean Time To Recovery) : temps moyen entre détection panne et service rétabli
- Taux de détection : % de pannes réelles détectées par le Monitor dans les 15 s
- Faux positifs : actions déclenchées à tort (ex : pic de latence transitoire)
- Overhead MAPE-K : consommation CPU/mémoire du gestionnaire autonome
- Nombre de cycles MAPE-K avant récupération complète

Présentez ces métriques sous forme de tableau comparatif :

Sans MAPE-K (intervention manuelle simulée) vs Avec MAPE-K (automatique)

3. Livrables attendus

Livable	Description	Pondération
L1 — Code source	Projet Maven (mesh-services, maape-k, chaos) + fichiers .proto	40 %
L2 — Rapport PDF	Architecture MAPE-K, métriques mesurées, analyse des limites, diagrammes (8-12 pages)	30 %
L3 — Démonstration	Présentation live 10 min : démarrage mesh → panne → récupération automatique	20 %
L4 — Qualité du code	Javadoc, gestion d'exceptions, thread-safety, lisibilité	10 %

4. Barème détaillé — /100 points

Critère	Description	Points
gRPC services	3 services opérationnels avec HealthCheck gRPC fonctionnel	15 pts
Sondes IPC	Sondes mesurent latence + statut via socket et transmettent à Monitor	10 pts
Monitor (M)	Reçoit et stocke correctement les mesures dans la KnowledgeBase	10 pts
Analyze (A)	Détecte pannes + dégradations selon seuils définis	15 pts
Plan & Execute (PE)	Actions correctives (≥ 2 types) effectivement exécutées	15 pts
ChaosMonkey	Injection de pannes, mesure MTTR avant/après MAPE-K	10 pts
Rapport PDF	Métriques mesurées, tableau	15 pts

	comparatif, analyse critique	
Démonstration	Démonstration fluide, maîtrise du système et des concepts	10 pts

⚠ La boucle MAPE-K doit être implémentée en Java pur — aucun framework d'orchestration externe (Kubernetes, Docker Swarm...) n'est autorisé.